

MEDIATEK

MT8668 Hypervisor Vcodec User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	刘志民	正式版

目录

版本记录 2

目录 3

图片目录 4

表格目录 4

1 视频编解码基本介绍 5

 1.1 目的 5

 1.2 范围 5

 1.2.1 缩略词 5

 1.2.2 性能 5

 1.3 架构/流程概要 7

 1.3.1 软件架构 7

 1.4 配置/客制指南 8

 1.4.1 解码器 codec 配置 8

 1.4.2 解码器支持路数配置 9

 1.4.3 Secure 解码器配置 9

 1.4.4 编码器 codec 配置 9

 1.4.5 编码器支持路数配置 10

 1.4.6 Secure 编码器配置 10

 1.4.7 Playback 功能客制化 11

 1.5 常见问题/故障排除 11

 1.5.1 MTK C2_hal CMD 11

 1.5.2 MMDVFS CMD 12

 1.5.3 VCodec CMD 12

 1.5.4 Video Dump 功能 12

 1.5.5 黑屏 13

 1.5.6 花屏 13

 1.5.7 卡顿 14

附件一 附加条款 15

图片目录

图 1-1. Android 系统 playback 架构 7

图 1-2. Hypervisor 系统 playback 架构 8

图 1-3. Hypervisor 系统 SVP playback 架构 8

表格目录

表 1-1. 缩略词 5

表 1-2. Video 文件格式 6

表 1-3. Vcodec 性能 6

1 视频编解码基本介绍

1.1 目的

本架构设计文档旨在描述具体的系统架构，以确保 Android 和 Yocto 系统能够在虚拟化环境中有效地共享硬件资源，同时向用户提供连贯和高性能的体验。

1.2 范围

本文档着重描述系统的高层架构，涵盖了从硬件层到应用层的设计考虑，以及不同组件之间的通信和互动关系。

1.2.1 缩略词

表 1-1. 缩略词

缩略词	全称	释义
HAL	Hardware Abstraction Layer	硬件抽象层
HDR	High Dynamic Range	高动态范围技术
Mbps	Megabits Per Second	兆比特每秒
SVP	Secure Video Playback	安全视频播放
V4L2	Video for Linux Two	Linux 系统 video 框架
VCodec	Video Codec	视频编解码器
VCP	Video CoProcess	Video 协处理器
Vdec	Video Decoder	视频解码器
Venc	Video Encoder	视频编码器
VirtIO	Virt Input/Output	虚拟输入输出

1.2.2 性能

下表是目前系统中支持的视频文件格式。

表 1-2. Video 文件格式

文件格式	文件后缀	MT8668 Android B0	备注
MKV	“.mkv”, “.mka”	√	-
MP4	“.mp4”, “.m4a”, “.m4v”, “.3g2”, “.3gp”, “.3gpp”, “.3gpp2”	√	-
PS	“.mpg”	√	vob 不支持
TS	“.ts”, “.m2ts”, “.mts”	√	-
WEBM	“.webm”	√	-
AVI	“.avi”	X	
WMV	“.wmv”, “.asf”	X	
FLV	“.flv”, “.f4v”	X	
RMVB	“.rmvb”, “.rm”	X	-

针对视频的编解码模块，下表是具体支持的参数列表信息。

表 1-3. Vcodec 性能

类别	编解码器	规格				
		分辨率	帧率	位速率	位深	属性/级别
解码器	H.264	4096x2176	30	100 Mbps	8	CBP, MP, HP/ 5.1
	H.265	4096x2176	30	100 Mbps	8/10	Main / 5.0 Main 10 / 5.0
	VP9	4096x2176	30	100 Mbps	8/10	Profile 0/2
编码器	H.264	3840x2160	30	100 Mbps	8	BP, MP, HP/ 5.1
	H.265	3840x2160	30	100 Mbps	8/10	Main, Main10 / 5.0
并发 (硬件性能)	-	4K30 VDEC + 4K30 VENC 或等效吞吐量				

1.3 架构/流程概要

1.3.1 软件架构

本章节主要介绍目前系统中 video playback 相关的架构图。

1.3.1.1 Android

在单系统中，video playback 的架构如下图所示。

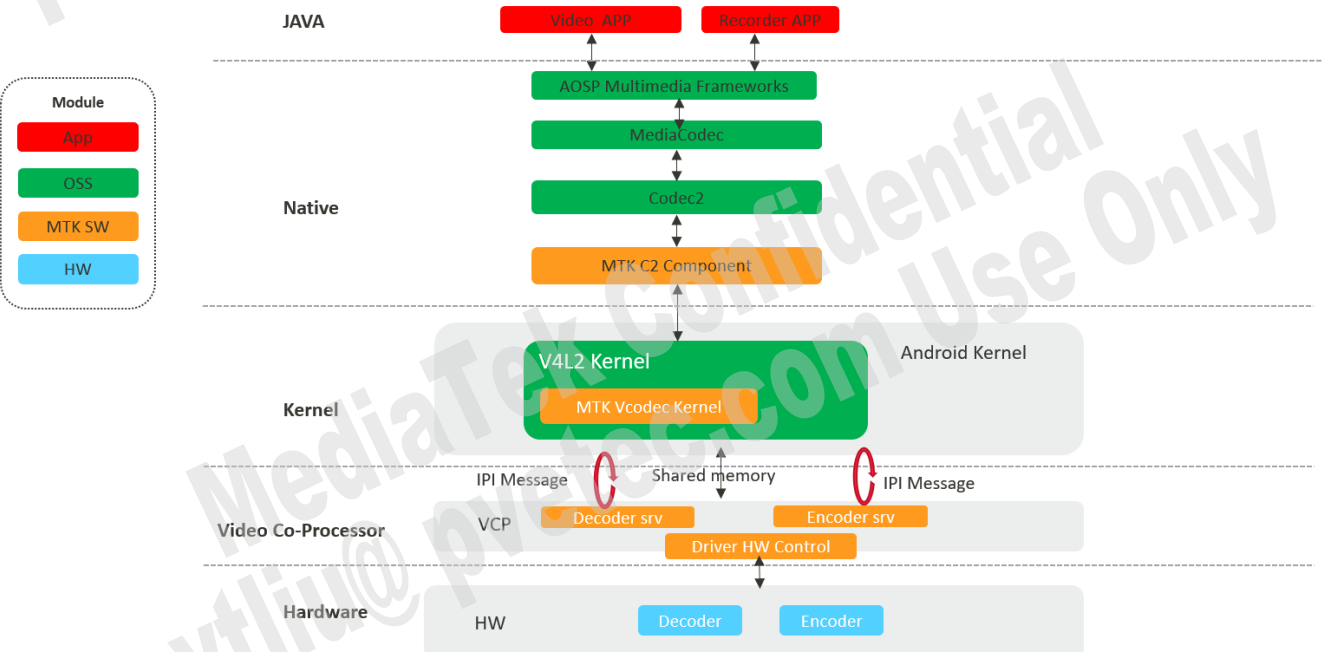


图 1-1. Android 系统 playback 架构

1.3.1.2 Hypervisor

在 Hypervisor 系统中，video playback 的架构如下图所示（该架构适用于 L(w/ Vcodec)+A 或者 L(w/ Vcodec)+L(w/o Vcodec)+A）。

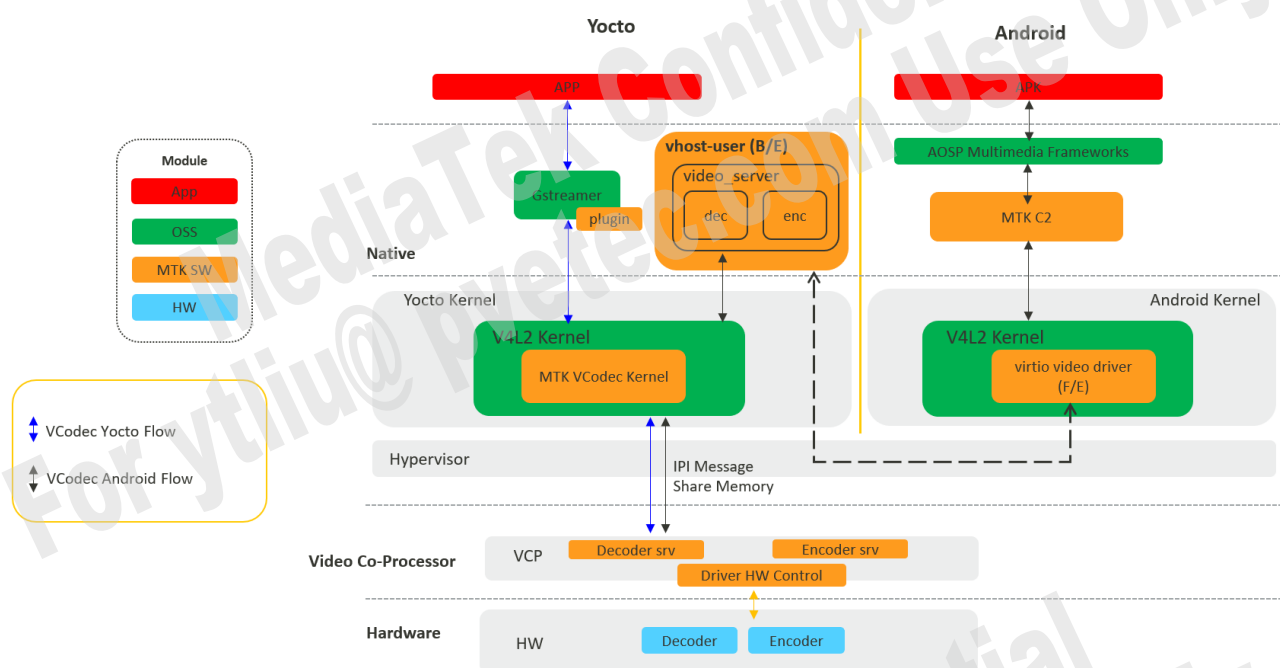


图 1-2. Hypervisor 系统 playback 架构

1.3.1.3 Hypervisor SVP

在 Hypervisor 系统中，secure video playback 的架构如下图所示。

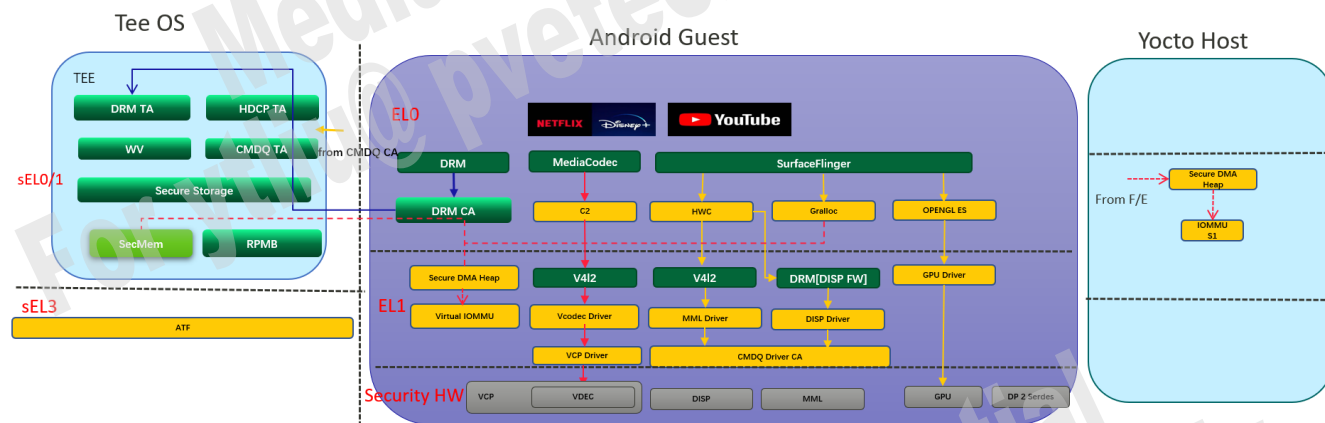


图 1-3. Hypervisor 系统 SVP playback 架构

1.4 配置/客制指南

1.4.1 解码器 codec 配置

下图是系统中配置的解码器支持的 codec 信息。

```

media_codecs_c2_svp.xml
113 <MediaCodec name="c2.mtk.avc.decoder" type="video/avc">
114   <Alias name="OMX.MTK.VIDEO.DECODER.AVC" />
115   <Limit name="size" min="64x64" max="4096x2176" />
116   <Quirk name="wants-NAL-fragments" />
117   <Feature name="adaptive-playback"/>
118   <Limit name="concurrent-instances" max="15" />
119   <Limit name="performance-point-3840x2160" value="30" />
120 </MediaCodec>

```

1.4.2 解码器支持路数配置

下图是系统中配置的解码器支持路数信息。

```

<MediaCodec name="c2.mtk.avc.decoder" type="video/avc">
  <Alias name="OMX.MTK.VIDEO.DECODER.AVC" />
  <Limit name="size" min="64x64" max="4096x2176" />
  <Quirk name="wants-NAL-fragments" />
  <Feature name="adaptive-playback"/>
  <Limit name="concurrent-instances" max="15" />
  <Limit name="performance-point-3840x2160" value="30" />
</MediaCodec>

```

1.4.3 Secure 解码器配置

下图是系统中支持 secure 解码器的配置信息。

```

<MediaCodec name="c2.mtk.avc.decoder.secure" type="video/avc">
  <Alias name="OMX.MTK.VIDEO.DECODER.AVC.secure" />
  <Limit name="size" min="64x64" max="1920x1088" />
  <Quirk name="wants-NAL-fragments" />
  <Feature name="secure-playback" required="true" />
  <Feature name="adaptive-playback"/>
  <Limit name="concurrent-instances" max="1" />
  <Limit name="performance-point-1920x1088" value="30" />
</MediaCodec>

```

1.4.4 编码器 codec 配置

下图是系统中支持的编码器对应的配置信息。

```

<MediaCodec name="c2.mtk.avc.encoder" type="video/avc" >
  <Alias name="OMX.MTK.VIDEO.ENCODER.AVC" />
  <Limit name="size" min="160x128" max="3840x2176" />
  <Limit name="alignment" value="2x2" />
  <Limit name="block-size" value="16x16" />
  <!-- max 3840x2160@60 -->
  <Limit name="block-count" range="80-32640" />
  <Limit name="blocks-per-second" range="80-1044480" />
  <Limit name="concurrent-instances" max="10" />
  <Limit name="bitrate" range="1-100000000" />
  <Limit name="performance-point-3840x2160" value="30" />
</MediaCodec>

```

1.4.5 编码器支持路数配置

下图是系统中编码器支持的通道数的配置信息。

```

<MediaCodec name="c2.mtk.avc.encoder" type="video/avc" >
  <Alias name="OMX.MTK.VIDEO.ENCODER.AVC" />
  <Limit name="size" min="160x128" max="3840x2176" />
  <Limit name="alignment" value="2x2" />
  <Limit name="block-size" value="16x16" />
  <!-- max 3840x2160@60 -->
  <Limit name="block-count" range="80-32640" />
  <Limit name="blocks-per-second" range="80-1044480" />
  <Limit name="concurrent-instances" max="10" />
  <Limit name="bitrate" range="1-100000000" />
  <Limit name="performance-point-3840x2160" value="30" />
</MediaCodec>

```

1.4.6 Secure 编码器配置

下图是系统中支持 secure 编码器的配置信息。

```

<MediaCodec name="c2.mtk.hevc.encoder.secure" type="video/hevc" >
  <Limit name="size" min="160x128" max="3840x2160" />
  <Limit name="alignment" value="16x16" />
  <Limit name="block-size" value="16x16" />
  <Limit name="quality" range="0-100" default="50" />
  <Limit name="concurrent-instances" max="10" />
  <Feature name="bitrate-modes" value="VBR,CBR,CQ" />
  <Limit name="performance-point-3840x2160" value="30" />
  <Feature name="special-codec" required="true" />
</MediaCodec>

```

1.4.7 Playback 功能客制化

1.4.7.1 HDR

关于 HDR 功能，需要具体项目进行客制化支持。

1.4.7.2 SVP Playback

关于 secure video playback 功能，需要具体项目进行客制化支持。

1.5 常见问题/故障排除

1.5.1 MTK C2_hal CMD

C2MtkComponent:

Enable all C2MTK ALOGV logs of C2MtkComponent

```
adb shell "setprop vendor.mtk.c2.enable.comp.log 2"
```

C2MtkVdec:

Enable C2MTK ALOGV logs of C2MtkVdec

```
adb shell "setprop vendor.mtk.c2.enable.vdec.log 2"
```

C2MtkVenc:

Enable C2MTK ALOGV logs of C2MtkVenc

```
adb shell "setprop vendor.mtk.c2.enable.venc.log 2"
```

C2MtkBufferManager:

Enable C2MTK ALOGV logs of C2MtkBufferManager

Enable C2MTK ALOGV internal logs of C2MtkBufferManager

```
adb shell "setprop vendor.mtk.c2.enable.bm.log 2"
```

```
adb shell "setprop vendor.mtk.c2.enable.bm.internal.log 1"
```

VCodec:

Enable all debug level logs of VCodec

Enable debug level logs by <TAG> filter. e.g., V4L2Device

```
adb shell "setprop vendor.mtk.c2.enable.vcodec.log.V4L2Device 4"
```

```
adb shell "setprop vendor.mtk.c2.enable.vcodec.log 4"
```

```
adb shell "setprop vendor.mtk.c2.enable.vcodec.log.<TAG> 4"
```

1.5.2 MMDVFS CMD

Force clock to specific step by OPP

param1: 0→Vcore 1→vmm

OPP 0: Highest clock rate

```
adb shell "echo 0 0 > /sys/module/mtk_mmdvfs_debug/parameters/force_step"
```

Vote clock to specific step by OPP:

```
adb shell "echo 0 0 > /sys/module/mtk_mmdvfs_debug/parameters/vote_step"
```

Dump current clk:

```
adb shell "echo fmeter > /proc/clkgdb ; cat /proc/clkgdb | grep -e vdec -e cam"
```

Dump MMDVFS OPP:

```
adb shell "cat /proc/mmdvfs/mmdvfs_opp"
```

1.5.3 VCodec CMD

Vcodec log Enable/Disable

1: Enable

2: Disable

```
adb shell "echo 1 > /sys/module/mtk_vcodec_dec/parameters/mtk_vcodec_dbg"
```

Vcodec log level control

codec_log: Common codec log level (0~31)

vpud_log: vcodec wrap log level (0~7)

job_log: Job log level (0~7)

```
adb shell "echo -codec_log 7 -vpud_log 3 -job_log 3 > /sys/module/mtk_vcodec_dec/parameters/mtk_vdec_vcp_log"
```

1.5.4 Video Dump 功能

Decoder input dump (bit-stream to decode)

The dumped file is saved in \data\vendor\vcodec\vdec_input_<timestamp>_<instance>.bs

e.g., \data\vendor\vcodec\vdec_input_20200716T133851.521_0xE80D6000.bs

```
adb shell "setprop vendor.mtk.c2.vdec.dump.input 1"
```

Decoder output dump (decoded video frames)

The dumped file is saved in:

\data\vendor\vcodec\vdec_output_<timestamp>_<instance>_W<width>H<height>.yuv

e.g., \data\vendor\vcodec\vdec_output_20200716T133851.521_0xE80D6000_W1280H720.yuv

```
adb shell "setprop vendor.mtk.c2.vdec.dump.output 1"
```

Decoder CC output dump (decoded video frames after color convert)

The dumped file is saved in:

```
\data\vendor\vcodec\vdec_output_cc_<timestamp>_<instance>_W<width>H<height>.yuv  
e.g., \data\vendor\vcodec\vdec_output_cc_20200716T133851.521_0xE80D6000_W1280H720.yuv  
adb shell "setprop vendor.mtk.c2.vdec.dump.output.cc 1"
```

Decoder profiling data dump (binary debugging log)

The dumped file is saved in \data\vendor\vcodec\vdec_profiling_<timestamp>_<instance>.dat

```
e.g., \data\vendor\vcodec\vdec_profiling_20200716T133851.521_0xE80D6000.dat  
adb shell "setprop vendor.mtk.c2.vdec.profiling 1"
```

1.5.5 黑屏

- 确认视频文件是否正常
- 确认正常启动视频解码模块

```
adb shell "setprop vendor.mtk.c2.enable.vdec.log 2"
```

在 logcat 日志中确认是否有类似下面的信息：

```
CCodec : allocate(c2.mtk.vp9.decoder)  
C2MtkComponentStore: find: name=c2.mtk.vp9.decoder, key=c2.mtk.vp9.decoder  
C2MtkVdec: [0xB400007D63859720] IntfImpl: name=c2.mtk.vp9.decoder
```

- 确认解码数据正常

确认解码模块有收到正常的 video 码流数据：

```
adb shell "setprop vendor.mtk.c2.vdec.dump.input 1"
```

- Force GPU 是否 OK

```
adb shell service call SurfaceFlinger 1008 i32 1
```

- 确认 display 模块显示正常

参考 display 文档确认显示逻辑。

如果以上几点都 OK，最后就需要 Video playback owner 详细查看 log 来定位问题。

1.5.6 花屏

- 确认解码数据正常确认解码模块有收到正常的 video 码流数据：

```
adb shell "setprop vendor.mtk.c2.vdec.dump.input 1"
```

- Force GPU 是否 OK

```
adb shell service call SurfaceFlinger 1008 i32 1
```

- 确认 display 模块显示正常

参考 display 文档确认显示逻辑。

如果以上几点都 OK，最后就需要 Video playback owner 详细查看 log 来定位问题。

1.5.7 卡顿

- 确认视频文件是否正常
- 确认正常启动视频解码模块

```
adb shell "setprop vendor.mtk.c2.enable.vdec.log 2"
```

在 logcat 日志中确认是否有类似下面的信息：

```
CCodec : allocate(c2.mtk.vp9.decoder)
C2MtkComponentStore: find: name=c2.mtk.vp9.decoder, key=c2.mtk.vp9.decoder
C2MtkVdec: [0xB400007D63859720] IntfImpl: name=c2.mtk.vp9.decoder
```

- 确认 display 模块显示正常

参考 display 文档确认显示逻辑。

如果以上几点都 OK，最后就需要 Video playback owner 详细查看 log 来定位问题。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。